- optimize code review for human time and effort
- with agents writing code it becomes almost impossible to review manually
- code review serves 2 purposes:
  A) DETECT AND CORRECT BUGS
  B) DEVELOP A SHARED UNDERSTANDING OF THE CODEBASE
- traditionally code review after the fact: waiting for someone to take a look
  —> with agents, integrate code review into the workflow, like a spell check continuously done
  This catches almost all the routine bugs (A) and reserves human attention for (B) and more high-level considerations
  Note: if we reduce review time but bugs still occur and take time to be fixed we are not solving the problem

- CLAUDE-CODE SUES WORKFLOW

  BEFORE PR
  - i) work is verified by Claude running tests and linter, asking to show the result
  - ii) review of diff by another agent with fresh context ) would not include
  - iii) required CI (lint, types, tests across platforms, build and packaging) => open findings block merge
  - iv) Agents hunt for bugs and review in parallel (against knowledge base)
  - v) verifiers vote on each finding, survivors ranked by severity } open findings block merge
  - vi) Separate security review ) optional

  need to set spending CAPs

  => RISK BASED APPROVAL: classifier sets the tier: no person review, fix pa (up to 2), human review

  MERGING
  - i) check run again against current develop and adpts merge queue

  need to set -> spending CAP

  AFTER THE MERGE
  - i) monitor for bugs / post-hoc findings => ISSUE creator for fix forwards / reverts
  - ii) fast rollback mechanism

  need -> to set Frequency

  => few bugs get through all the layers, if some do, we have a fast rollback in place

- Keep CLAUDE.md brief and reference the rest of the dos
  AGENTS.md
- REVIEW.md here one can specify what matters in their part of the code:
  - define project failure
  - limit nitpicking
  - define ignored paths (could be updated / moved to knowledge base)
  - require file + line reference
  - not guessing filenames
  - require top level summary
  - try to find root cause always

  IMPORTANT: Nit preexisting bugs to avoid unfairly penalizing the author for bugs that were not introduced by the PR

- VOTERS.md shown only to verifier to not be gamified
  - findings about style, diagnostics, cleanups are filtered out (A/B tested by Anthropic)
  - other rules from AGENT.md

- RISK TIERS ARE INFLUENCED BY FILETYPE:
  - docs, copy, additive changes for already implemented code => NO HUMAN
  - ordinary changes, sensitive changes, system prompts, user defined => HUMAN

- always put feat/, bugfix/ flags on PRs

- collect incidents and update rubrics for evaluation

- stated priorities 1) DETECT REAL BUGS
  2) REDUCING NUMBER OF REVIEW CYCLES

## WHERE HUMANS IN THE LOOP ARE NEEDED

- deciding whether or not to build something
- setting vision, taste, overall goals for the product
- auditing risky code from (A)
- verifying outputs (end product)

## ABOUT (B): UNDERSTANDING THE CODEBASE

IF the bulk of the review / diff reading is done by machines, what becomes the way to learn the codebase? Do humans even need to learn it?

ANTHROPIC SWEs STILL DO NOT HAVE A FORMAL MECHANISM

——> no one needs a full mental model: rather, when someone wants to learn something about a piece of the codebase, they ask Claude to explain it

——> The coding workflow becomes:

human involved:
1) ASK EXPLANATION
2) DEFINE CHANGE AND ASK FEEDBACK / QUESTIONS
3) REVIEW FEEDBACK AND PROMPT GOAL

4) LET AGENTS IMPLEMENT

human involved only for high risk changes ->
5) AUTOMATED REVIEW AND MERGE

human
6) REVIEW AFTER PRODUCT COMPLETED